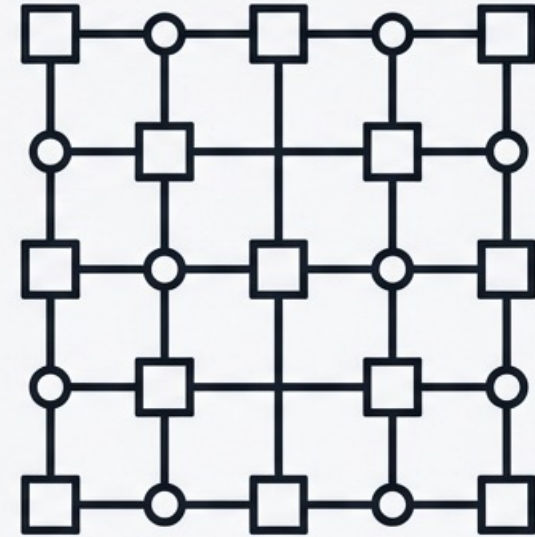


Introdução à Orientação a Objetos

Da abstração do mundo real à execução na Máquina Virtual Java.



O Mundo Real



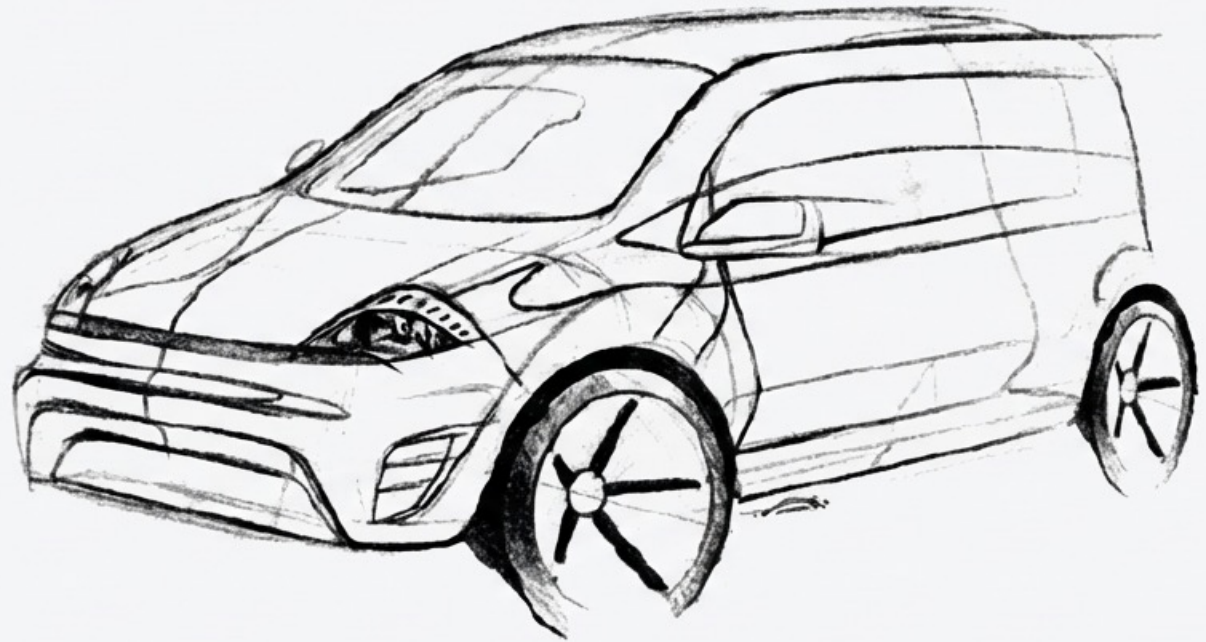
O Mundo Virtual

- **O Paradigma:** Programação Orientada a Objetos (POO) simula as entidades e interações do mundo real dentro do software.
- **O Objetivo:** Aumentar produtividade, encapsular complexidade e promover o reuso de código através de componentes isolados que se comunicam via mensagens.

A Classe: O Rascunho

Uma classe é a descrição de um conjunto de entidades. Ela não existe fisicamente no sistema; é apenas um modelo, um protótipo ou uma ideia.

Ao criar a classe Carro, definimos apenas o que ele tem (propriedades) e o que ele faz (comportamentos). Não definimos a cor exata ou quem é o dono.



O Objeto: A Instância Real

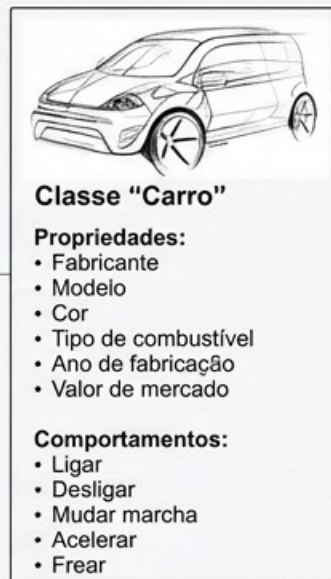
Quando usamos a classe para fabricar algo, criamos um objeto. O objeto ocupa espaço na **memória**, possui **identidade única** e um **estado próprio**.

Este Ford Fiesta prata é uma instância. Ele acelera de verdade, tem um chassi único e seus atributos possuem valores concretos.



Síntese do Mapeamento

Define a estrutura universal (fabricante, modelo, cor, métodos).



Múltiplas instâncias independentes coexistindo na memória, cada uma com seus próprios dados.

Criando uma Classe na Prática

```
class Carro {  
    String fabricante;  
    String modelo;  
    String cor;  
    int anoDeFabricacao;  
    double valorDeMercado;  
}
```

A declaração da estrutura (o tipo).

Os tipos de dados que definem o estado do objeto (Variáveis de Instância).

Instanciação: A Palavra-chave new

```
Carro meuCarro = new Carro();
```

Declaração. Cria uma variável capaz de armazenar uma referência para um objeto do tipo Carro.

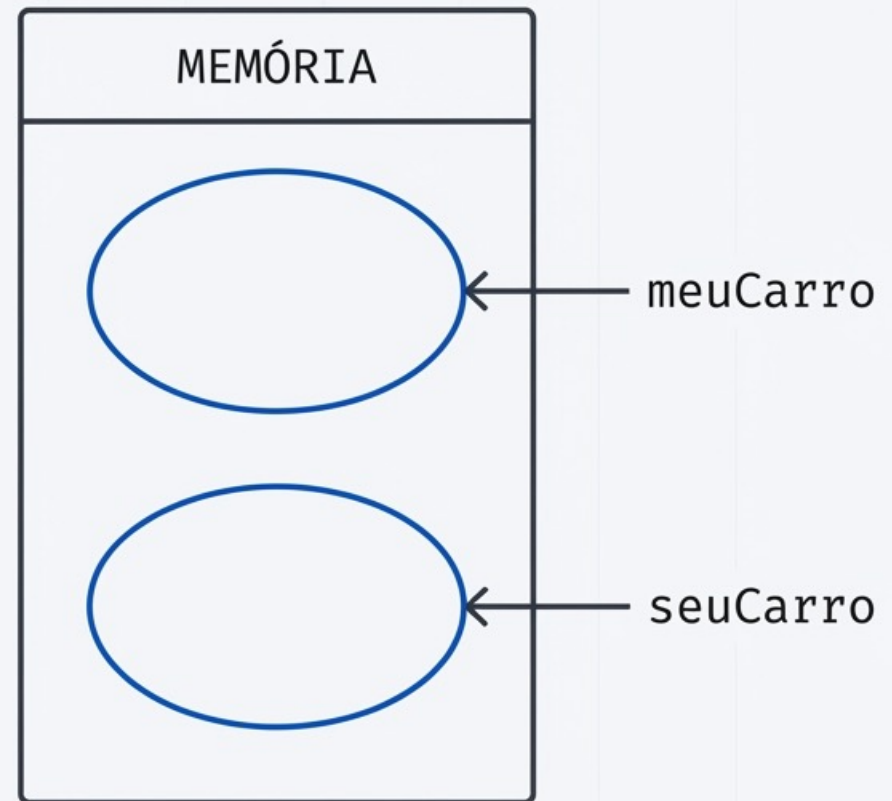
Atribuição. Conecta a variável ao objeto recém-criado.

Instanciação. A JVM aloca espaço na memória para um novo objeto físico.

O Objeto na Memória

A execução de `new Carro()` aloca blocos distintos na memória da Máquina Virtual Java.

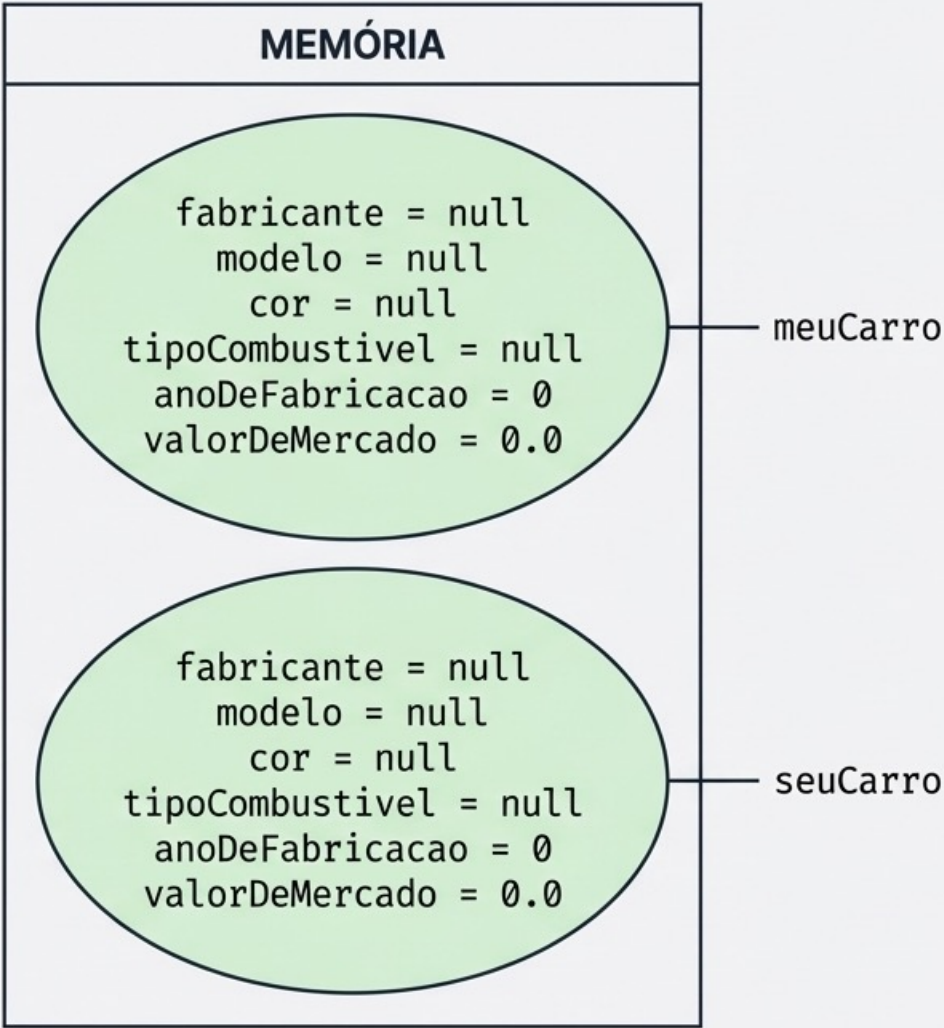
As variáveis `meuCarro` e `seuCarro` são apenas ponteiros. Elas não guardam o carro, elas apontam para o endereço dele.



Valores Padrão (Defaults)

Tipo	Valor Padrão
Tipos Numéricos (int, double)	0 ou 0.0
Tipo Booleano (boolean)	false
Referências (String, Objetos)	null

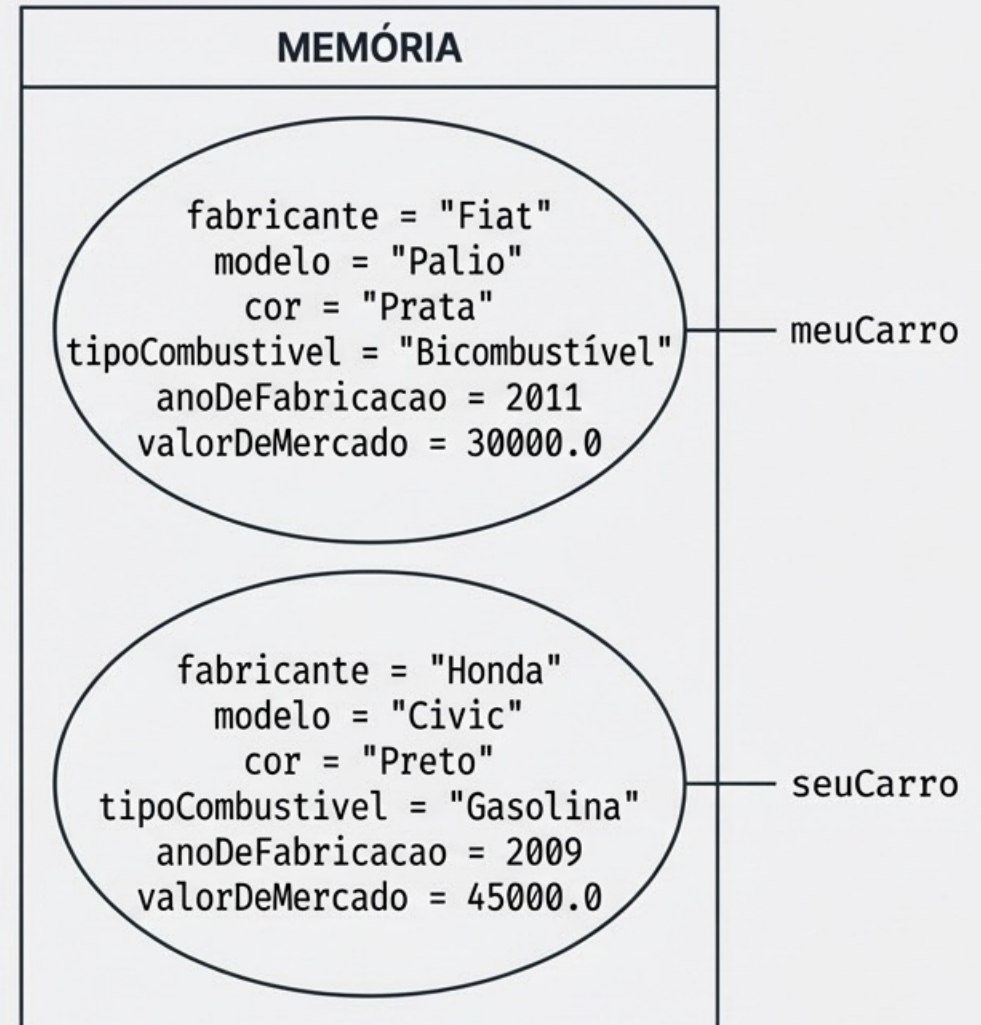
Se não atribuirmos valores, a JVM preenche a memória automaticamente com estes padrões para evitar lixo de memória.



Acessando o Estado (A Notação de Ponto)

```
meuCarro.anoDeFabricacao = 2011;  
meuCarro.modelo = "Palio";  
seuCarro.modelo = "Civic";
```

O operador . (ponto) é a chave de acesso. Ele diz à JVM: Siga esta referência até a memória e modifique/leia este atributo específico.



O Problema da Baixa Coesão

```
class Carro {  
    String modelo;  
    // ...  
    String nomeProprietario;  
    String cpfProprietario;  
    String logradouroProprietario;  
}
```

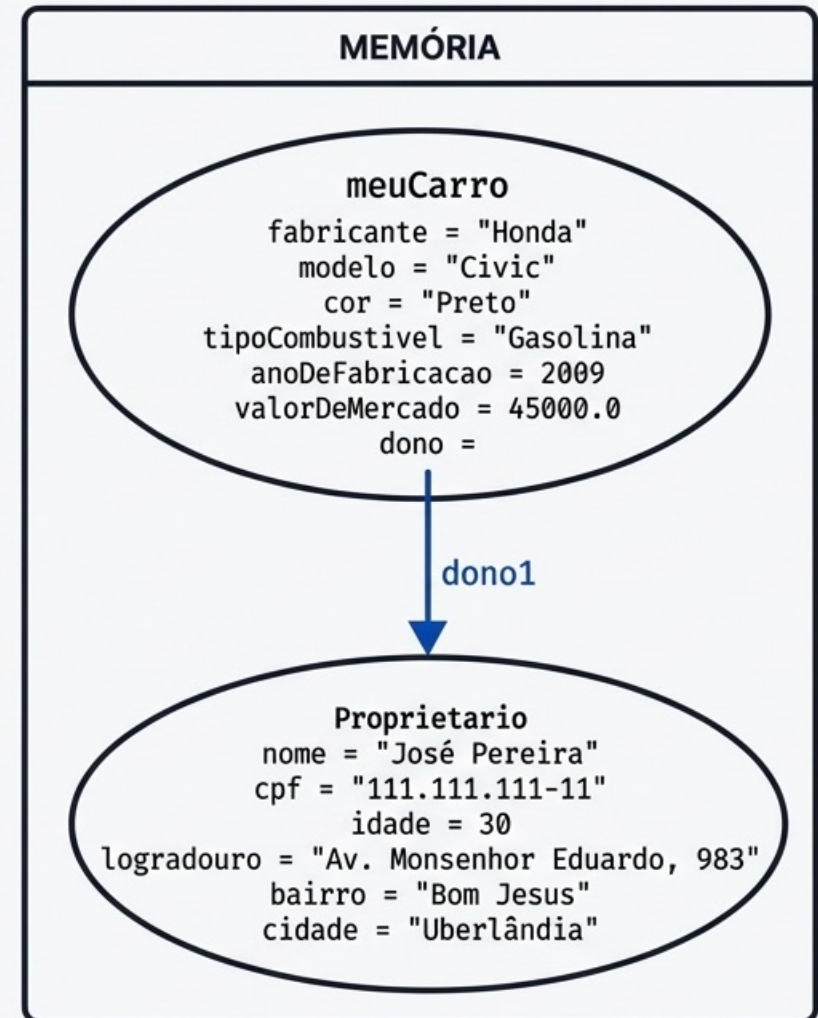


Um carro real não possui CPF ou Logradouro. Inserir estes dados na classe Carro cria uma arquitetura frágil com excesso de **responsabilidades**. A classe perde **coesão**.

Composição de Objetos ("Tem um")

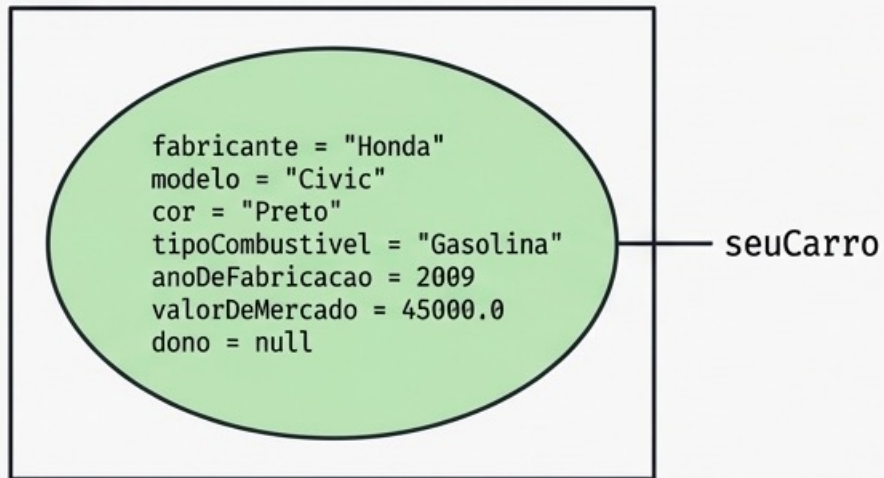
A solução é delegar. Criamos uma classe Proprietario independente e colocamos uma referência a ela dentro de Carro. O objeto composto agora se alinha perfeitamente com a lógica do mundo real.

```
meuCarro.dono = dono1;
```



O Perigo

memória

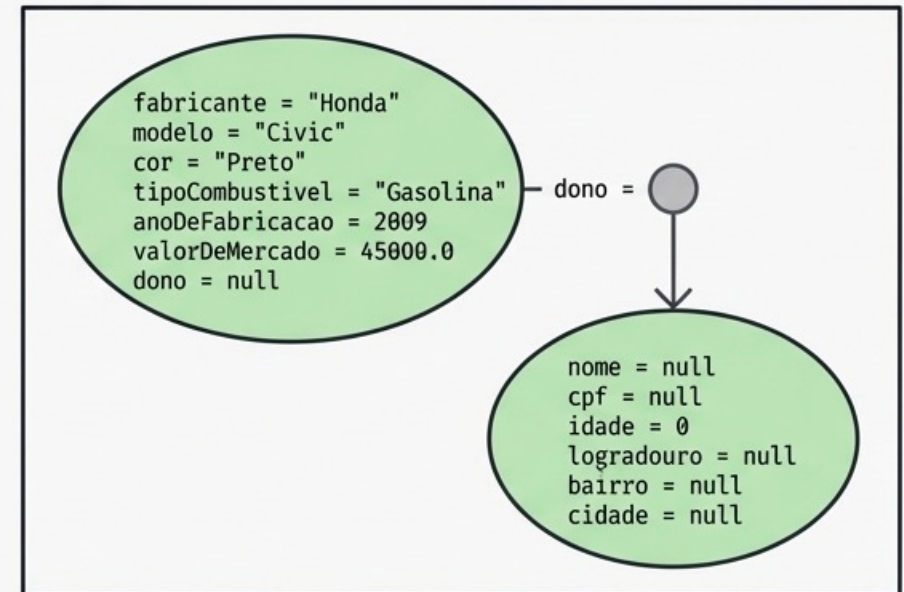


```
seuCarro.dono.nome = "José"; // ERRO!
```

Tentar acessar um atributo de uma referência **null** quebra o programa.

A Solução

memória

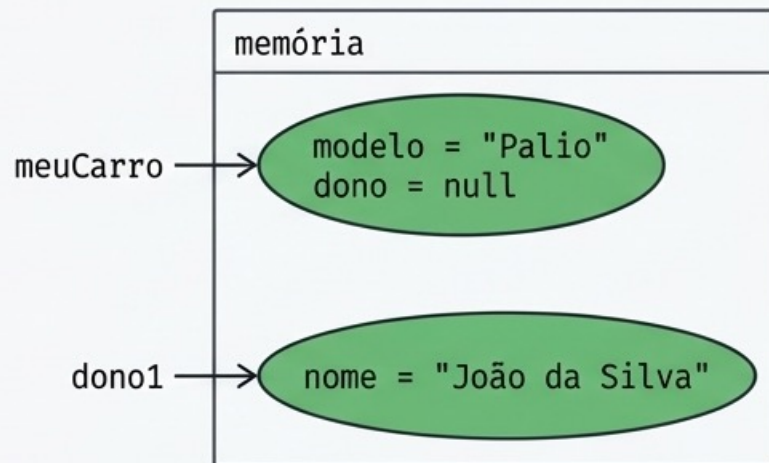


```
seuCarro.dono = new Proprietario();
```

O objeto dependente deve ser instanciado antes de ser acessado.

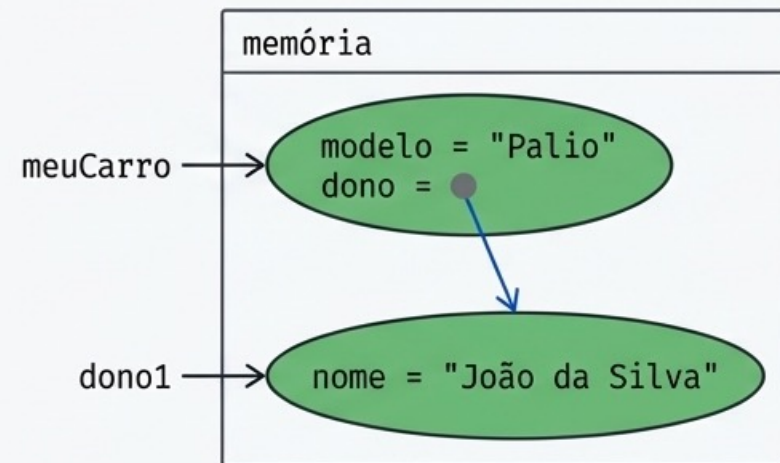
Regra de Ouro: Variáveis são Controles Remotos

PASSO 1



Dois objetos existem isoladamente na memória.

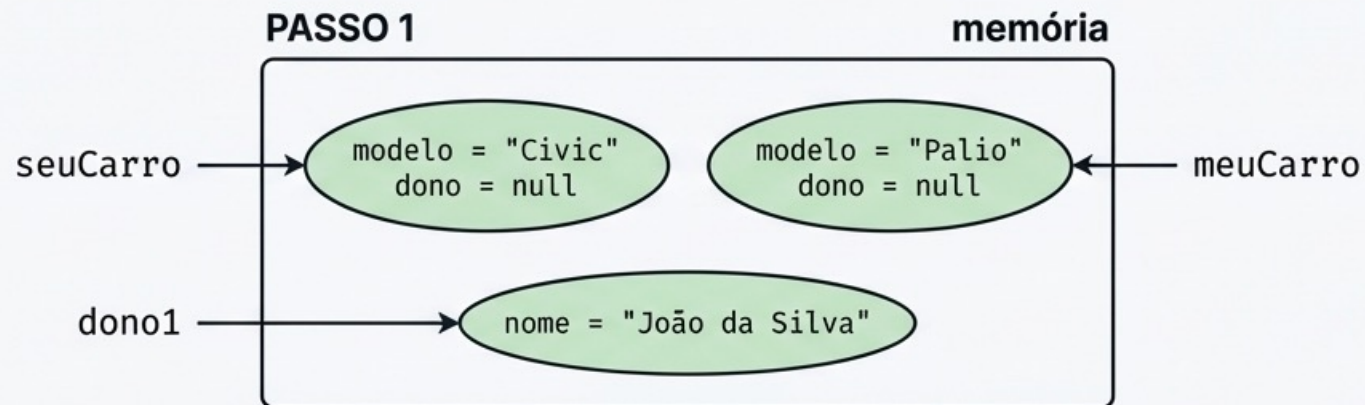
PASSO 2



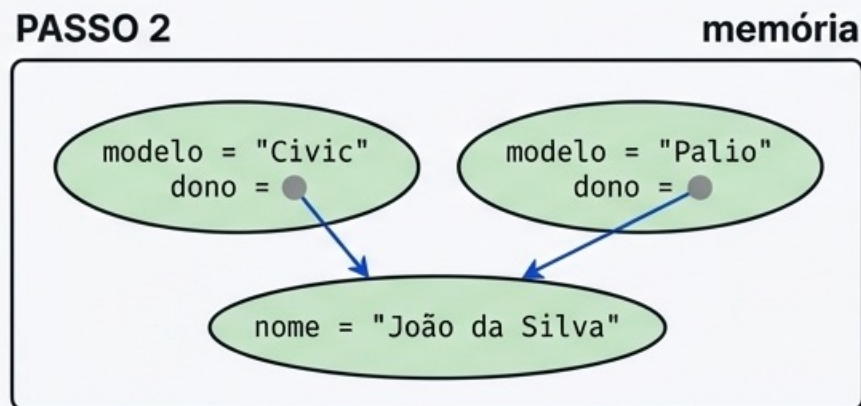
```
meuCarro.dono = dono1;
```

A variável não copia o objeto; ela estabelece um cabo de conexão (ponteiro) até ele.

Múltiplas Referências, Um Único Objeto

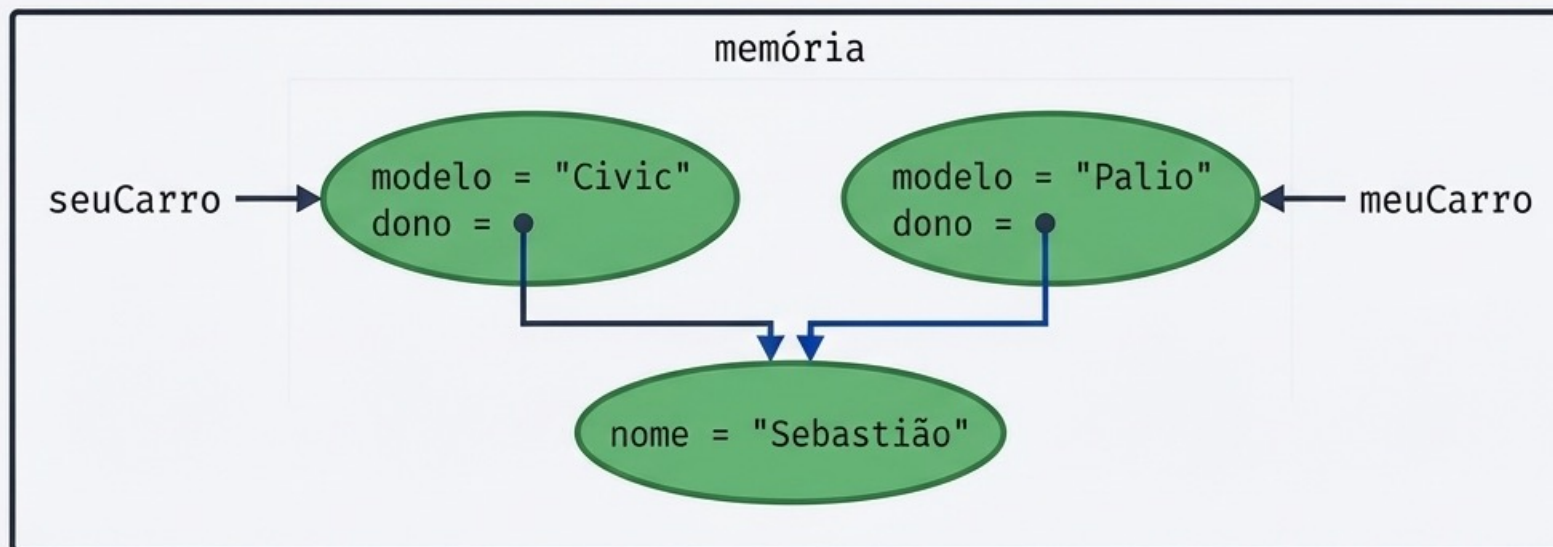


```
meuCarro.dono = dono1;  
seuCarro.dono = dono1;
```



O João ("dono1") possui dois carros. Ambos os controles remotos apontam para exatamente o mesmo endereço de memória.

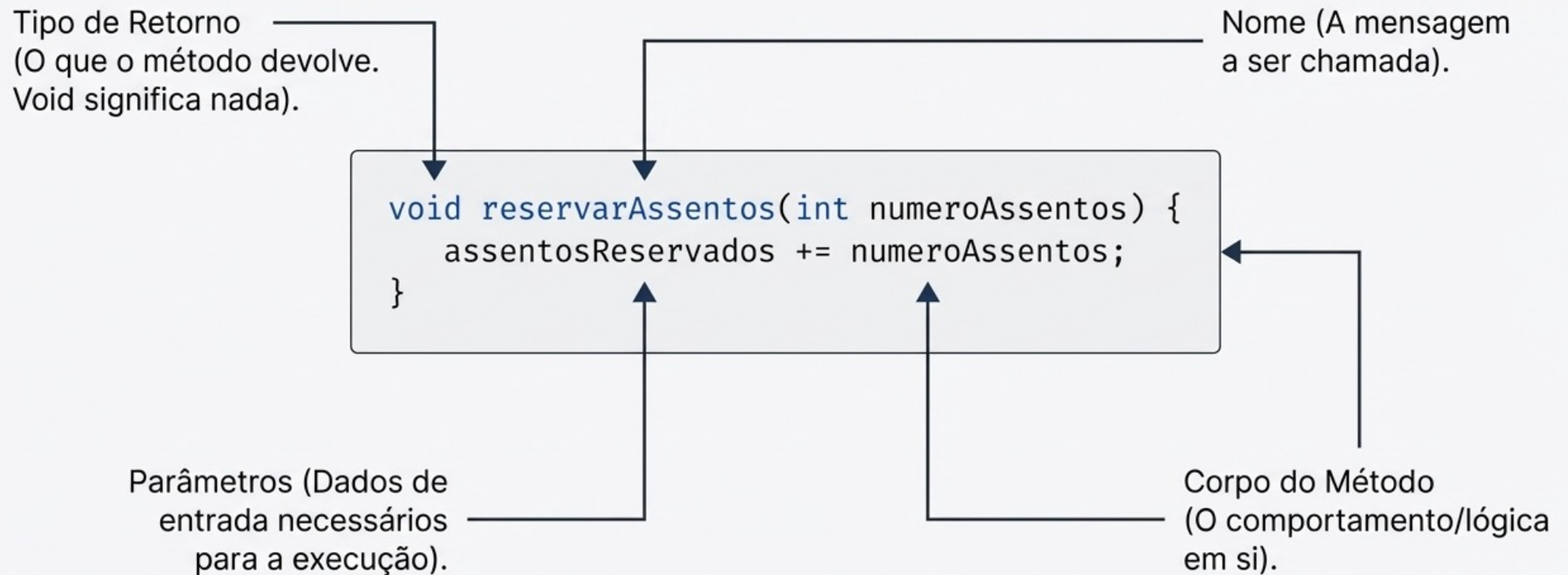
O Impacto da Referência Compartilhada



```
meuCarro.dono.nome = "Sebastião";
```

Alterar o nome via `meuCarro` altera simultaneamente para `seuCarro`. Por quê? Porque existe apenas um dono físico na memória. Se o João muda de nome, ele muda para todos os seus carros.

Anatomia de um Método



Passagem de Argumentos: Primitivos vs. Objetos

Tipos Primitivos (int, double)	Objetos (Carro, Proprietario)
Passagem por Valor (Pass-by-value). A JVM cria uma cópia do dado. Alterações dentro do método não afetam a variável original externa.	Passagem por Referência (Pass-by-reference). A JVM passa uma cópia do controle remoto. Alterações feitas dentro do método modificam objeto original na memória.

A palavra-chave this

```
this.assentos += assentos;
```

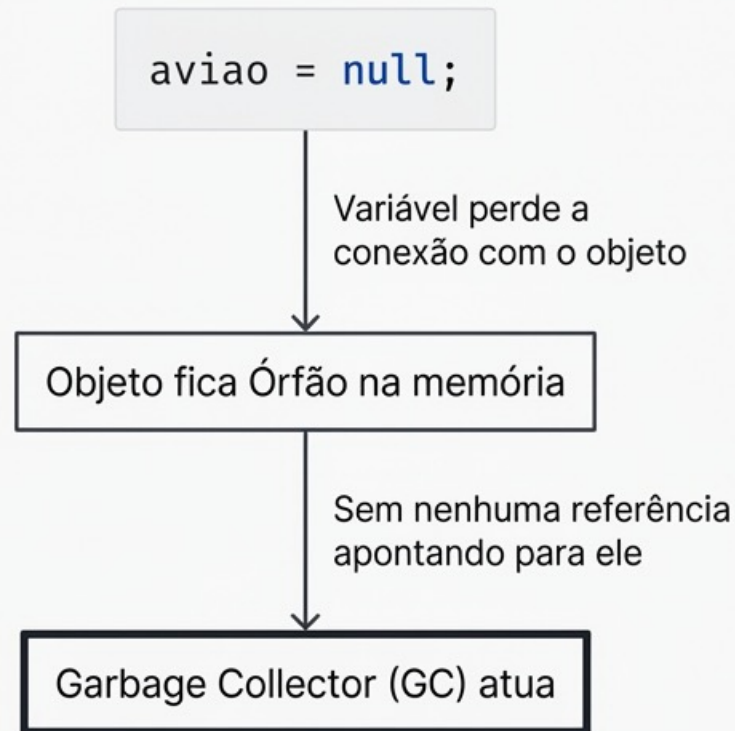
this é uma auto-referência. Ele resolve ambiguidades, dizendo à JVM: Use a variável da minha própria instância, não o parâmetro que acabei de receber.

Sobrecarga (Overloading)

```
void reservar(int qtd) { ... }  
void reservar(int normais, int  
especiais) { ... }
```

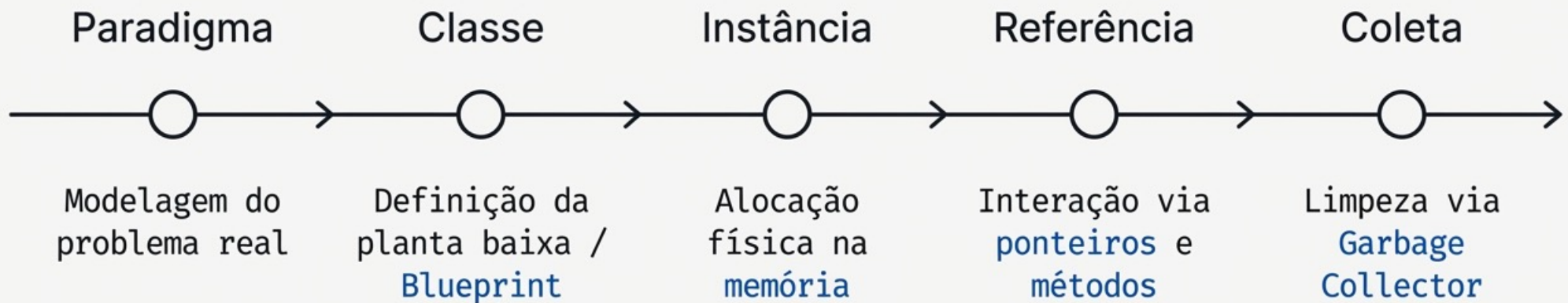
Múltiplos métodos com o mesmo nome, mas assinaturas (parâmetros) diferentes. A JVM sabe qual executar baseada nos argumentos enviados.

Ciclo de Vida e o Coletor de Lixo



O Java desobriga o programador de limpar a memória manualmente. O GC é um processo em segundo plano que varre a JVM destruindo objetos inacessíveis e devolvendo o espaço livre.

O Ecossistema da Orientação a Objetos



Fim do Capítulo 8. Documentação Técnica Minimalista.